



Trabalho Prático

Grupo C

Resolução de Problemas com Grafos

INTEGRANTES

-  Igor Praciano Thomaz
-  Rafael Lima Cacau
-  Tiago Silveira Taumaturgo

Unidade 3 - T1 | Problema: Treehouses

O Problema Treehouses (Kattis)

? O que precisamos resolver?

- Conectar todas as casas na árvore com o menor comprimento total de cabos.
- Aproveitar as conexões que já existem sem custo extra.

i Contexto do Desafio

- ✓ Algumas casas já estão conectadas à rede principal.
- ✓ Pares específicos já possuem cabos instalados.

Objetivo Final

Calcular o **custo adicional mínimo** necessário para a conectividade total.

Modelagem como Grafo

Componentes do Grafo



Vértices

Cada Treehouse é um nó no grafo



Arestas

Possíveis conexões entre as casas



Pesos

Distância física entre as coordenadas

DISTÂNCIA EUCLIDIANA

$$\text{dist}(i, j) = \sqrt{((x_i - x_j)^2 + (y_i - y_j)^2)}$$

Grafo não direcionado, ponderado e completo (K_n)

A Variação do MST

Conexões Gratuitas

- ✓ **Rede Inicial:** As primeiras e treehouses já estão interconectadas.
- ✓ **Cabos Existentes:** p pares informados já possuem cabos.

Essas conexões entram no grafo com:

PESO 0.0

O Diferencial

Não buscamos o custo total de conectar tudo do zero.

A resposta é o **menor comprimento adicional** de cabos necessário.

Estratégia Algorítmica

1

Inicialização de Conexões

Adicionar arestas de **peso zero** para as primeiras e casas e os p pares existentes.

2

Construção do Grafo Completo

Calcular a distância euclidiana para todos os outros pares de treehouses possíveis.

3

Ordenação Global

Ordenar todas as arestas por **peso crescente** para o processamento do Kruskal.

4

Kruskal + Union-Find (DSU)

Iterar pelas arestas e unir componentes se não formarem ciclos, garantindo o **custo mínimo**.

Implementação em Java - Parte 1

```
7 public class Main {  
8  
9     Run | Debug  
10    public static void main(String[] args) throws Exception {  
11        java.io.File file = new java.io.File("dados/dados02.txt");  
12        if (file.exists()) {  
13            System.setIn(new java.io.FileInputStream(file));  
14        }  
15  
16        int n = StdIn.readInt();  
17        int e = StdIn.readInt();  
18        int p = StdIn.readInt();  
19  
20        double[] x = new double[n];  
21        double[] y = new double[n];  
22  
23        for (int i = 0; i < n; i++) {  
24            x[i] = StdIn.readDouble();  
25            y[i] = StdIn.readDouble();  
26        }  
27  
28        EdgeWeightedGraph graph = new EdgeWeightedGraph(n);
```

Leitura das coordenadas e inicialização do grafo

Implementação em Java - Parte 2

```
27
28     for (int i = 1; i < e; i++) {
29         graph.addEdge(new Edge(v: 0, i, weight: 0.0));
30     }
31
32     for (int i = 0; i < p; i++) {
33         int a = StdIn.readInt() - 1;
34         int b = StdIn.readInt() - 1;
35         graph.addEdge(new Edge(a, b, weight: 0.0));
36     }
37
38     for (int i = 0; i < n; i++) {
39         for (int j = i + 1; j < n; j++) {
40             double dx = x[i] - x[j];
41             double dy = y[i] - y[j];
42             double dist = Math.sqrt(dx * dx + dy * dy);
43             graph.addEdge(new Edge(i, j, dist));
44         }
45     }
46
47     KruskalMST mst = new KruskalMST(graph);
48
49     System.out.printf(Locale.US, "%.10f%n", mst.weight());
50 }
```

Código

Complexidade da Solução

Etapa do Algoritmo	Complexidade	Descrição
Geração de Arestas	$O(n^2)$	Cálculo de distâncias para o grafo completo.
Ordenação (Kruskal)	$O(m \log m)$	Onde $m \approx n^2/2$ arestas totais.
Union-Find (DSU)	$O(m \alpha(n))$	Operações de união e busca quase lineares.

COMPLEXIDADE TEMPORAL TOTAL

$O(n^2 \log n)$

COMPLEXIDADE ESPACIAL

$O(n^2)$

Casos Especiais e Implementação

⚙️ Detalhes Técnicos

- **Conversão de Índices:** A entrada do Kattis usa base 1, convertida para base 0 no Java.
- **Precisão Decimal:** Saída obrigatória com 10 casas decimais para aceitação.

```
System.out.printf("%.10f\n", totalCost);
```

⚠️ Casos de Borda

- **Conectividade Inicial:** Se as conexões gratuitas já unem todas as casas, o custo é 0.0.
- **Grafo Completo:** Tratamento de $N(N-1)/2$ arestas para garantir a otimalidade.
- **Linguagem:** Implementação robusta utilizando a classe Scanner e tipos double.

ACCEPTED

Conclusão e Resultados

Problema Treehouses resolvido com sucesso.

- 💡 A modelagem de conexões gratuitas como arestas de peso zero simplifica a aplicação do algoritmo de Kruskal.
- ✅ O uso de Union-Find (DSU) garante eficiência no controle de componentes e ciclos.
- 📈 A complexidade $O(n^2 \log n)$ mostrou-se ideal para os limites de tempo do Kattis.

ENTREGÁVEIS DO PROJETO:

Código Java

README Técnico

Apresentação PDF